

ReproLab Database Design and Data Architecture

Updated with Prisma Authentication Tables

Abdul Moiz Hussain

ReproLab Engineering

Contents

1 Document Scope	3
2 Storage Boundaries and Design Principles	3
2.1 What not to store in PostgreSQL	3
3 Authentication and Identity Tables	4
3.1 User	4
3.2 Session	5
3.3 Account	6
3.4 Verification	7
4 ReproLab Domain Tables	7
4.1 GitHub Installations	7
4.2 Projects	8
4.3 Environment Specs	8
4.4 Scenarios	9
4.5 Runs	9
4.6 Run Attempts	10
4.7 Runner Nodes	10
4.8 Artifacts	11
4.9 Outbox	11
5 Environment Specs vs Scenarios	11
6 Artifact Usage	12
7 Transactional Outbox Pattern	12
8 Entity Relationships	13
9 State Management and Lease Rules	13
10 Retention and Archival Policy	13
11 Security Notes	14
11.1 Authentication tables	14
11.2 GitHub credentials	14
11.3 Artifacts	14

12 Recommended Indexes	14
13 Appendix: Prisma Schema	16

1 Document Scope

This document describes the durable PostgreSQL data model for ReproLab based on the *ReproLab Polyglot Construction Roadmap*, updated to include the Prisma / Better Auth identity tables:

- `user`
- `session`
- `account`
- `verification`

The original roadmap defines a conceptual `users` table. In this updated design, the Prisma `user` table is treated as the canonical user table. ReproLab domain tables that reference a user therefore reference `user.id`.

The Prisma authentication tables handle product login, sessions, linked OAuth accounts, and verification tokens. They are distinct from ReproLab’s GitHub App installation records, which remain stored in `github_installations`.

2 Storage Boundaries and Design Principles

ReproLab uses different storage systems for different kinds of data.

Table 1: Storage responsibility matrix

System	Owns	Must not own
PostgreSQL	Durable product metadata, auth users, sessions, accounts, verification records, projects, runs, attempts, leases, artifact metadata, outbox events.	High-volume raw logs, videos, trace archives, large HTTP bodies, raw GitHub installation tokens.
Redis Streams	Durable-enough work delivery and low-latency live events.	Canonical long-term artifact storage.
S3-compatible object storage	Immutable run bundles, trace files, videos, screenshots, compressed logs, manifests, checksums.	Run authorization logic.

2.1 What not to store in PostgreSQL

The following should not be stored as normal PostgreSQL rows:

- Every log line from a long run.
- Video, screenshots, trace archives, or compressed bundles.
- Raw GitHub App installation tokens.
- Full unredacted environment variables.
- Large HTTP bodies by default.
- Plaintext passwords or long-lived secrets.

Small searchable summaries, manifests, checksums, and metadata belong in PostgreSQL. The complete immutable evidence belongs in object storage.

3 Authentication and Identity Tables

The following tables are managed by Prisma. They use camelCase column names and lowercase singular table names through `@@map`.

These tables handle:

- User accounts,
- Login sessions,
- OAuth or credential accounts,
- Email or identifier verification tokens.

3.1 User

Table 2: user — Prisma model User

Field	Type	Description / Constraints
<code>id</code>	TEXT	Primary key. Prisma CUID: <code>@id @default(cuid())</code> .
<code>email</code>	TEXT	Required and unique. Primary login identity.
<code>name</code>	TEXT	Nullable. Display name. This corresponds to the roadmap's conceptual <code>display_name</code> .
<code>emailVerified</code>	BOOLEAN	Required. Default <code>false</code> . Indicates whether the email address has been verified.
<code>image</code>	TEXT	Nullable. Avatar or profile image URL.
<code>createdAt</code>	TIMESTAMPZ	Required. Default <code>now()</code> .
<code>updatedAt</code>	TIMESTAMPZ	Required. Automatically maintained by Prisma using <code>@updatedAt</code> .

Prisma mapping: `@@map("user")`.

Relations: `sessions` and `accounts` are Prisma relation fields, not physical columns.

3.2 Session

Table 3: session — Prisma model `Session`

Field	Type	Description / Constraints
<code>id</code>	TEXT	Primary key. Session identifier.
<code>expiresAt</code>	TIMESTAMPTZ	Required. Time when the session expires.
<code>token</code>	TEXT	Required. Unique session token. Enforced by <code>@@unique([token])</code> .
<code>createdAt</code>	TIMESTAMPTZ	Required. Default <code>now()</code> .
<code>updatedAt</code>	TIMESTAMPTZ	Required. Automatically maintained by Prisma using <code>@updatedAt</code> .
<code>ipAddress</code>	TEXT	Nullable. IP address associated with the session, if captured.
<code>userAgent</code>	TEXT	Nullable. User agent associated with the session, if captured.
<code>userId</code>	TEXT	Required. Foreign key to <code>user.id</code> . Cascade delete: <code>onDelete: Cascade</code> .

Prisma mapping: `@@map("session")`.

Indexes: `@@index([userId])`.

Uniqueness: `@@unique([token])`.

3.3 Account

Table 4: account — Prisma model Account

Field	Type	Description / Constraints
<code>id</code>	TEXT	Primary key. Internal account record identifier.
<code>accountId</code>	TEXT	Required. Identifier for the external account, for example a GitHub user ID.
<code>providerId</code>	TEXT	Required. Authentication provider identifier, for example <code>github</code> .
<code>userId</code>	TEXT	Required. Foreign key to <code>user.id</code> . Cascade delete: <code>onDelete: Cascade</code> .
<code>accessToken</code>	TEXT	Nullable. OAuth access token, if required by the auth flow. Should be encrypted or avoided when not needed.
<code>refreshToken</code>	TEXT	Nullable. OAuth refresh token, if required by the auth flow. Should be encrypted or avoided when not needed.
<code>idToken</code>	TEXT	Nullable. OpenID Connect ID token, if applicable.
<code>accessTokenExpiresAt</code>	TIMESTAMPZ	Nullable. Expiry time for the access token.
<code>refreshTokenExpiresAt</code>	TIMESTAMPZ	Nullable. Expiry time for the refresh token.
<code>scope</code>	TEXT	Nullable. OAuth scopes granted for this account.
<code>password</code>	TEXT	Nullable. Password credential for email/password login. Must store only a hashed password, never plaintext.
<code>createdAt</code>	TIMESTAMPZ	Required. Default <code>now()</code> .
<code>updatedAt</code>	TIMESTAMPZ	Required. Automatically maintained by Prisma using <code>@updatedAt</code> .

Prisma mapping: `@map("account")`.

Indexes: `@index([userId])`.

Recommended uniqueness: For typical OAuth login behavior, a composite unique constraint on (`providerId`, `accountId`) is recommended if each external account should map to only one user.

3.4 Verification

Table 5: verification — Prisma model `Verification`

Field	Type	Description / Constraints
<code>id</code>	TEXT	Primary key. Verification record identifier.
<code>identifier</code>	TEXT	Required. Identifier being verified, commonly an email address or user identifier.
<code>value</code>	TEXT	Required. Verification token, code, or signed value.
<code>expiresAt</code>	TIMESTAMPZ	Required. Expiry time for the verification token.
<code>createdAt</code>	TIMESTAMPZ	Required. Default <code>now()</code> .
<code>updatedAt</code>	TIMESTAMPZ	Required. Automatically maintained by Prisma using <code>@updatedAt</code> .

Prisma mapping: `@@map("verification")`.

Indexes: `@@index([identifier])`.

4 ReproLab Domain Tables

The following tables correspond to the durable data model described in the roadmap. The roadmap’s conceptual `users` table is implemented by the Prisma `user` table defined above.

4.1 GitHub Installations

Table 6: `github_installations`

Field	Type	Description / Constraints
<code>id</code>	TEXT	Primary key.
<code>user_id</code>	TEXT	Required. Foreign key to <code>user.id</code> . Recommended cascade delete.
<code>installation_id</code>	BIGINT	Required and unique. GitHub App installation ID.
<code>account_login</code>	TEXT	GitHub account or organization login name.
<code>suspended_at</code>	TIMESTAMPZ	Nullable. Set when the installation is suspended, revoked, or otherwise disabled.

Important distinction: This table stores GitHub App installation metadata for repositories. It is not the same as the Prisma `account` table, which stores user authentication provider accounts.

4.2 Projects

Table 7: projects

Field	Type	Description / Constraints
id	TEXT	Primary key.
owner_id	TEXT	Required. Foreign key to <code>user.id</code> .
repository_full_name	TEXT	Required. Repository full name, for example <code>org/repo</code> .
default_branch	TEXT	Required. Default branch, for example <code>main</code> .
visibility	TEXT	Required. Repository visibility. Recommended check constraint: <code>public</code> or <code>private</code> .

4.3 Environment Specs

Table 8: environment_specs

Field	Type	Description / Constraints
id	TEXT	Primary key.
project_id	TEXT	Required. Foreign key to <code>projects.id</code> . Recommended cascade delete.
version	INTEGER	Required. Version number of the environment specification.
object_key	TEXT	Required. Object-storage key for the full environment specification file.
schema_version	INTEGER	Required. Version of the ReproLab specification schema.
checksum	TEXT	Required. SHA-256 checksum of the stored specification.

Recommended uniqueness: `UNIQUE (project_id, version)`.

4.4 Scenarios

Table 9: scenarios

Field	Type	Description / Constraints
id	TEXT	Primary key.
project_id	TEXT	Required. Foreign key to <code>projects.id</code> . Recommended cascade delete.
name	TEXT	Required. Human-readable scenario name.
type	TEXT	Required. Scenario type, for example <code>actions</code> or <code>playwright_script</code> .
version	INTEGER	Required. Scenario version.
object_key	TEXT	Required. Object-storage key for the scenario definition or committed Playwright reference.
checksum	TEXT	Required. SHA-256 checksum of the stored scenario artifact.

Recommended uniqueness: `UNIQUE (project_id, name, version)` or `UNIQUE (project_id, id, version)` depending on how scenario names are managed.

4.5 Runs

Table 10: runs

Field	Type	Description / Constraints
id	TEXT	Primary key.
project_id	TEXT	Required. Foreign key to <code>projects.id</code> .
requested_commit	TEXT	Branch name, tag, or partial SHA originally requested by the user.
resolved_commit	TEXT	Exact immutable Git commit SHA used for the run.
status	TEXT	Run state. Example values: <code>QUEUED</code> , <code>LEASED</code> , <code>CLONING</code> , <code>BUILDING</code> , <code>STARTING</code> , <code>READY</code> , <code>RECORDING</code> , <code>COLLECTING</code> , <code>SUCCESS</code> , <code>FAILED</code> , <code>CANCELLED</code> , <code>ABANDONED</code> .
requested_by	TEXT	Required. Foreign key to <code>user.id</code> .
created_at	TIMESTAMPTZ	Required. Default <code>now()</code> .

4.6 Run Attempts

Table 11: run_attempts

Field	Type	Description / Constraints
id	TEXT	Primary key. This is the attempt identifier.
run_id	TEXT	Required. Foreign key to runs.id. Recommended cascade delete.
runner_id	TEXT	Foreign key to runner_nodes.id. Identifies the runner that owns the attempt.
lease_token	TEXT	Required for active attempts. Unique token used for compare-and-set state transitions.
lease_expires_at	TIMESTAMPTZ	Time when the lease expires if the runner heartbeat is not renewed.
status	TEXT	Attempt state, for example LEASED, CLONING, BUILDING, STARTING, RECORDING, COLLECTING, SUCCESS, FAILED, CANCELLED, ABANDONED.
started_at	TIMESTAMPTZ	Nullable. Time when execution started.
ended_at	TIMESTAMPTZ	Nullable. Time when execution reached a terminal state.

Idempotency rule: The unique execution identity is (run_id, attempt_id). A duplicate job delivery must resume or exit safely; it must never create a second independent attempt.

4.7 Runner Nodes

Table 12: runner_nodes

Field	Type	Description / Constraints
id	TEXT	Primary key.
name	TEXT	Runner hostname or machine identifier.
runtime_version	TEXT	Runner runtime version, sandbox runtime version, or build identifier.
capacity	INTEGER	Maximum concurrent attempts allowed on this runner. Initially recommended value: 1.
last_heartbeat_at	TIMESTAMPTZ	Last heartbeat timestamp. Used for lease expiry and recovery.
draining	BOOLEAN	Required. Default false . If true, the runner should not receive new jobs.

4.8 Artifacts

Table 13: artifacts

Field	Type	Description / Constraints
id	TEXT	Primary key.
attempt_id	TEXT	Required. Foreign key to <code>run_attempts.id</code> . Recommended cascade delete.
kind	TEXT	Artifact type. Examples: <code>manifest</code> , <code>events</code> , <code>trace</code> , <code>video</code> , <code>screenshot</code> , <code>network.har</code> , <code>service-log</code> , <code>metrics</code> , <code>checksums</code> .
object_key	TEXT	Required. Object-storage key for the immutable artifact file.
size_bytes	BIGINT	Required. Artifact size in bytes.
sha256	TEXT	Required. SHA-256 checksum for integrity verification.
content_type	TEXT	MIME type, for example <code>application/gzip</code> , <code>video/webm</code> , or <code>application/json</code> .

4.9 Outbox

Table 14: outbox

Field	Type	Description / Constraints
id	TEXT	Primary key.
aggregate_type	TEXT	Domain aggregate type, for example <code>Run</code> , <code>Project</code> , or <code>Attempt</code> .
aggregate_id	TEXT	Identifier of the domain aggregate.
event_type	TEXT	Event name, for example <code>RunCreated</code> , <code>RunCancelled</code> , or <code>AttemptCompleted</code> .
payload	JSONB	Event payload to be published to Redis Streams or another message transport.
published_at	TIMESTAMPTZ	Nullable. Timestamp when the event was successfully published. Null means pending.

Recommended partial index: `CREATE INDEX ON outbox (published_at) WHERE published_at IS NULL;`

5 Environment Specs vs Scenarios

ReproLab separates the environment definition from the browser scenario.

Table 15: Environment spec and scenario comparison

Concept	Purpose	Examples
Environment Spec	Defines the isolated application environment to build and launch.	Dockerfile, services, Postgres, Redis, ports, health checks, environment variable references, resource limits, capture settings.
Scenario	Defines the browser actions that reproduce the bug.	Login, click checkout, fill form, expect response, capture screenshot, assert URL, run committed Playwright script.

Simple distinction:

The **environment spec** builds the world.
The **scenario** acts inside that world.

Both must be versioned and checksummed so that reruns can prove exactly which environment and which scenario were used.

6 Artifact Usage

The **artifacts** table stores metadata about immutable evidence produced by a run attempt. The actual artifact files are stored in S3-compatible object storage.

Artifacts include:

- `manifest.json`,
- normalized event streams,
- Playwright trace archives,
- videos,
- screenshots,
- network HAR files,
- compressed service logs,
- container metrics,
- checksums.

Artifacts are used for:

1. Reconstructing a completed run without Redis.
2. Rendering the synchronized timeline.
3. Providing shareable read-only reproduction links.
4. Comparing failed and fixed commits.
5. Preserving immutable evidence with checksums.
6. Supporting retention and lifecycle policies.

PostgreSQL stores artifact metadata only. Object storage stores the heavy files.

7 Transactional Outbox Pattern

The **outbox** table ensures that database commits and message publication cannot diverge.

When Express creates a run, it writes both the run record and an outbox event inside the same PostgreSQL transaction:

1. Insert into `runs`.
2. Insert into `outbox`.
3. Commit the PostgreSQL transaction.
4. A background publisher reads pending outbox events.
5. The publisher appends the event to Redis Streams.
6. The publisher marks `published_at`.

This prevents the following failure:

The run is committed to PostgreSQL, but the Redis publish fails, so no runner ever receives the job.

With the outbox pattern, the run and the event are committed atomically. If Redis is temporarily unavailable, the publisher retries until the event is delivered.

Delivery should still be treated as at-least-once. Therefore, runners and event consumers must be idempotent.

8 Entity Relationships

The main relationships are:

- One `user` has many `session` records.
- One `user` has many `account` records.
- One `user` has many `github_installations`.
- One `user` owns many `projects`.
- One `user` can request many `runs`.
- One `project` has many `environment_specs`.
- One `project` has many `scenarios`.
- One `project` has many `runs`.
- One `run` has many `run_attempts`.
- One `runner_node` can execute many `run_attempts`.
- One `run_attempt` produces many `artifacts`.

9 State Management and Lease Rules

ReproLab run state transitions must be guarded.

Important rules:

1. A run is created transactionally before a job is published.
2. Every state transition uses compare-and-set semantics.
3. Only a `QUEUED` run can become `LEASED`.
4. Only the owning lease token can advance an attempt.
5. A runner must heartbeat regularly.
6. A lease expires if the heartbeat is not renewed.
7. A recovery worker may mark an attempt `ABANDONED`.
8. A job is acknowledged only after terminal status and artifact metadata are persisted.
9. Duplicate job delivery must not create duplicate attempts.

10 Retention and Archival Policy

Retention should be policy-driven and reconciled between PostgreSQL and object storage.

Recommended rules:

- Keep failed run artifacts longer than successful run artifacts by default.
- Allow project owners to pin important runs.
- Delete expired object-storage prefixes using lifecycle rules.
- Reconcile `artifacts` rows after object deletion.
- Store small searchable summaries in PostgreSQL.
- Store complete evidence in object storage.
- Expire verification tokens and sessions according to auth policy.

11 Security Notes

11.1 Authentication tables

- The `password` field must store only a strong password hash.
- OAuth tokens in `account` should be encrypted at rest if persisted at all.
- Session tokens must be treated as secrets.
- Verification tokens must expire and should be single-use.
- User deletion should cascade sessions and accounts.

11.2 GitHub credentials

- GitHub App installation tokens are short-lived and minted only when needed.
- Raw GitHub installation tokens should not be stored in PostgreSQL.
- The `github_installations` table stores installation metadata, not long-lived credentials.
- Credentials passed to runners should use encrypted job payloads or one-time secret references.

11.3 Artifacts

- Artifact access must be authorized on every request.
- Download URLs should be short-lived signed URLs.
- Secret-looking values must be redacted before events leave the runner.
- A second redaction pass should occur before final bundle publication.

12 Recommended Indexes

In addition to primary keys and explicit unique constraints, the following indexes are recommended:

- `user(email)`
- `session(userId)`
- `session(token)`
- `account(userId)`
- `account(providerId, accountId)` if provider accounts should be globally unique
- `verification(identifier)`
- `github_installations(user_id)`
- `github_installations(installation_id)` unique
- `projects(owner_id)`
- `projects(repository_full_name)`
- `environment_specs(project_id, version)` unique
- `scenarios(project_id, name, version)` unique or equivalent
- `runs(project_id, status, created_at)`
- `runs(resolved_commit)`

- `run_attempts(run_id)`
- `run_attempts(lease_expires_at)`
- `runner_nodes(last_heartbeat_at)`
- `artifacts(attempt_id, kind)`
- Partial index on `outbox(published_at)` where `published_at` IS NULL

13 Appendix: Prisma Schema

The following Prisma schema defines the authentication and identity tables included in this updated database design.

```
model User {
  id          String    @id @default(cuid())
  email       String    @unique
  name        String?
  emailVerified Boolean  @default(false)
  image       String?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
  sessions    Session[]
  accounts    Account[]

  @@map("user")
}

model Session {
  id          String    @id
  expiresAt   DateTime
  token       String
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
  ipAddress   String?
  userAgent   String?
  userId      String
  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@unique([token])
  @@index([userId])
  @@map("session")
}

model Account {
  id          String    @id
  accountId   String
  providerId  String
  userId      String
  user        User      @relation(fields: [userId], references: [id],
    onDelete: Cascade)
  accessToken String?
  refreshToken String?
  idToken     String?
  accessTokenExpiresAt DateTime?
  refreshTokenExpiresAt DateTime?
  scope       String?
  password    String?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  @@index([userId])
  @@map("account")
}

model Verification {
  id          String    @id
```



```
    identifier String
    value        String
    expiresAt    DateTime
    createdAt    DateTime @default(now())
    updatedAt    DateTime @updatedAt

    @@index([identifier])
    @@map("verification")
}
```